

디지털 취약 계층 대상

AI Agent 쇼핑 어시스턴트 서비스

뽕랑구

방세은 이소윤 임소정 원지우

2026 데이터종합분석 최종발표

Team 한살차이



목차

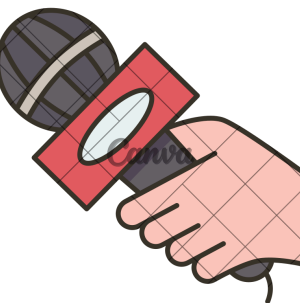
- 01 Introduction
- 02 Usecase
- 03 Demo
- 04 Technical Details
- 05 Challenges & Solutions
- 07 마무리

01

딸랑구를 기억하시나요?

효창공원에서
직접 인터뷰를 진행!

글자 크기가 크게 제일 좋은 거 같아





그러니까 요 글씨가 작으니까는



딸랑구의 목표

디지털 취약계층이 스스로 구매를 이어나갈 수 있도록

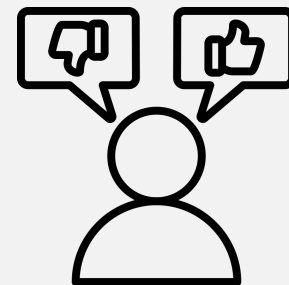
말로 요청하면



자연어 요청

“저번에 먹었던 딸기 사줘”처럼
사용자가 음성으로 요청하면
구매 의도를 추출

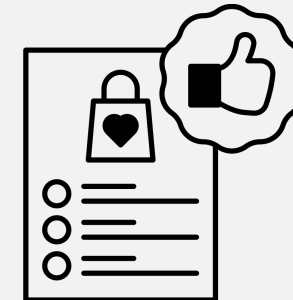
내 취향대로



구매 이력 기반 탐색

이전 구매 기록과
선호 정보를 바탕으로
관련 상품 후보를 탐색

알아서 골라



맥락 기반 후보 선정

사용자의 요청과 맥락을 해석,
구매 가능성이 높은
후보를 비교, 선정

구매까지!



TOI 기반 주문 실행

사용자의 최종 음성 승인 후
장바구니/ 주문 프로세스를
알아서 진행

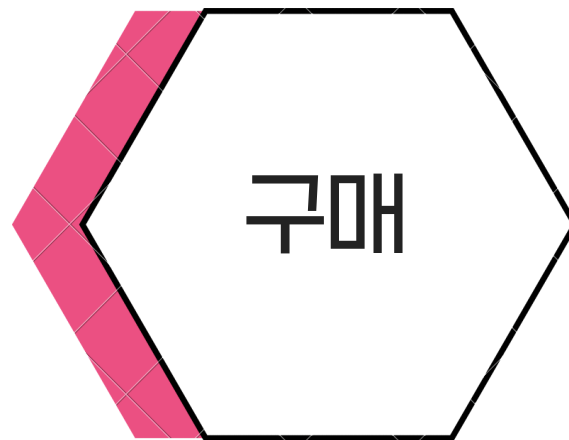
02

UseCase

Use Case Flow: 음성 기반 쇼핑 플로우

Agent를 설계하기에 앞서,
3가지의 해결하고자 하는 UseCase를 명확하게 설정

Case #1



Case #2



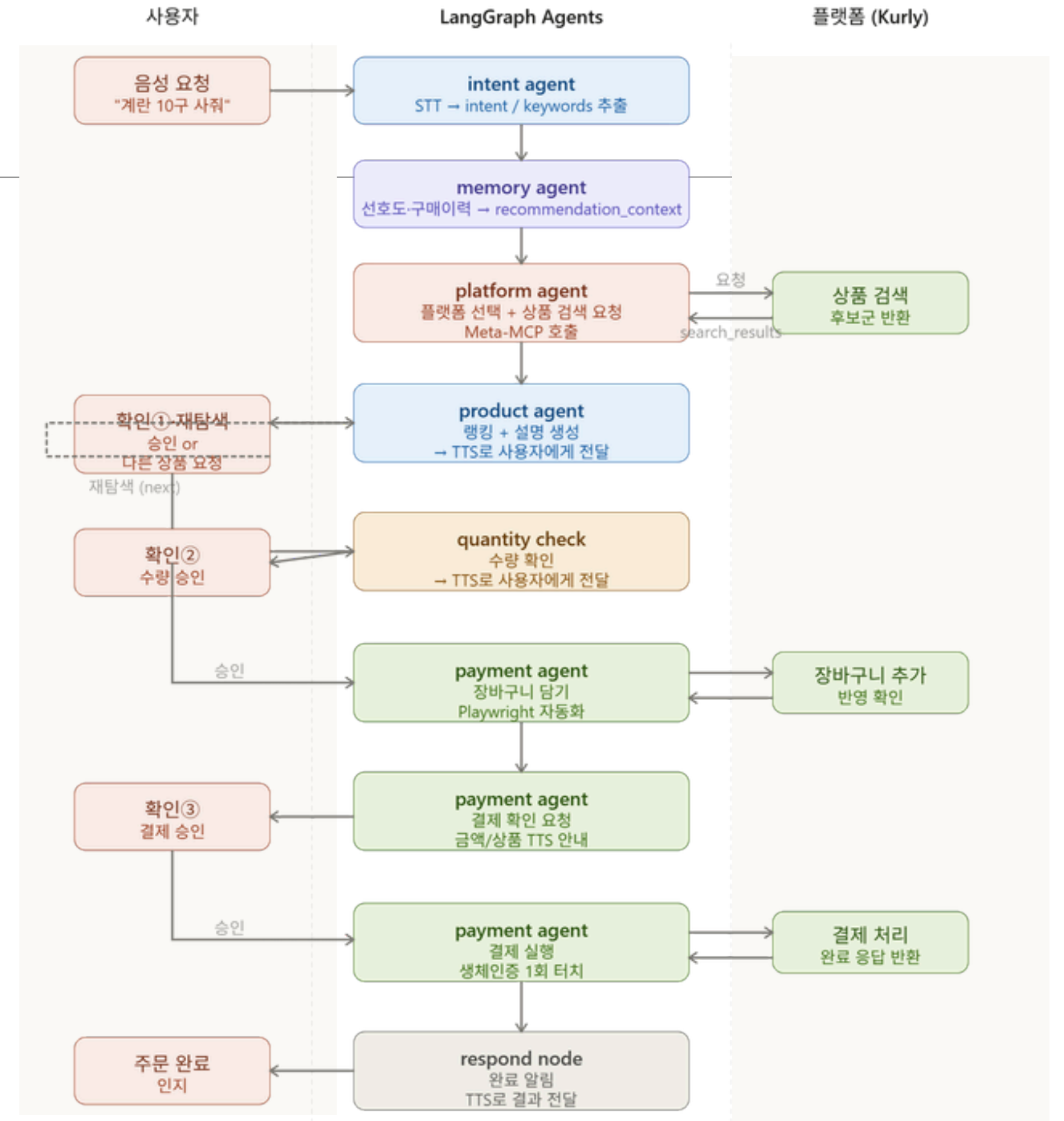
Case #3



Case #1 : 새로운 상품 구매

뭘 살지는 알겠는데, 어디서 어떻게
사야할지 모르겠다면?

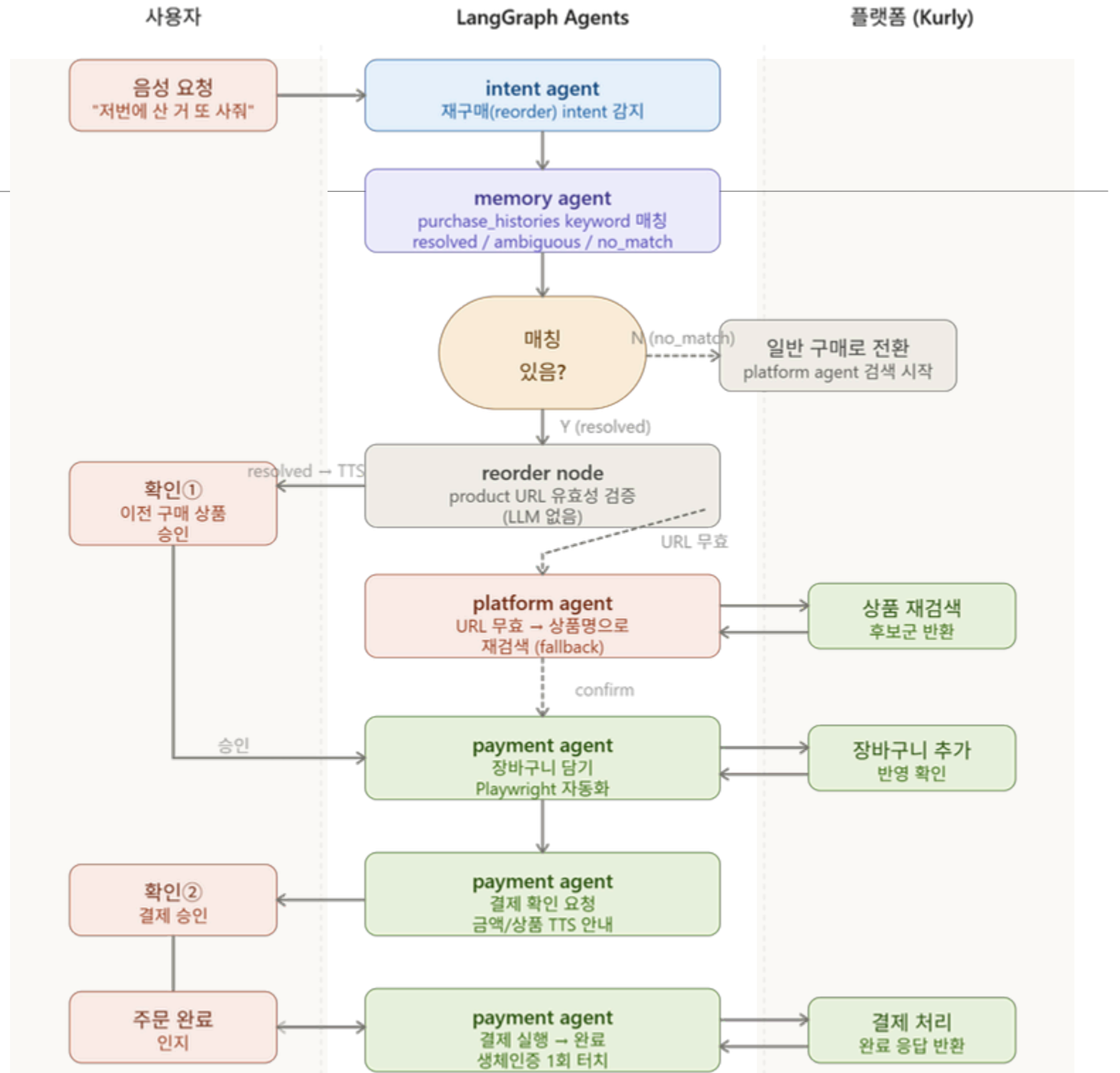
딸랑구가 해결!



Case #2 : 재구매

저번에 샀던 그거,
또 시키고 싶다면?

달랑구가 해결!

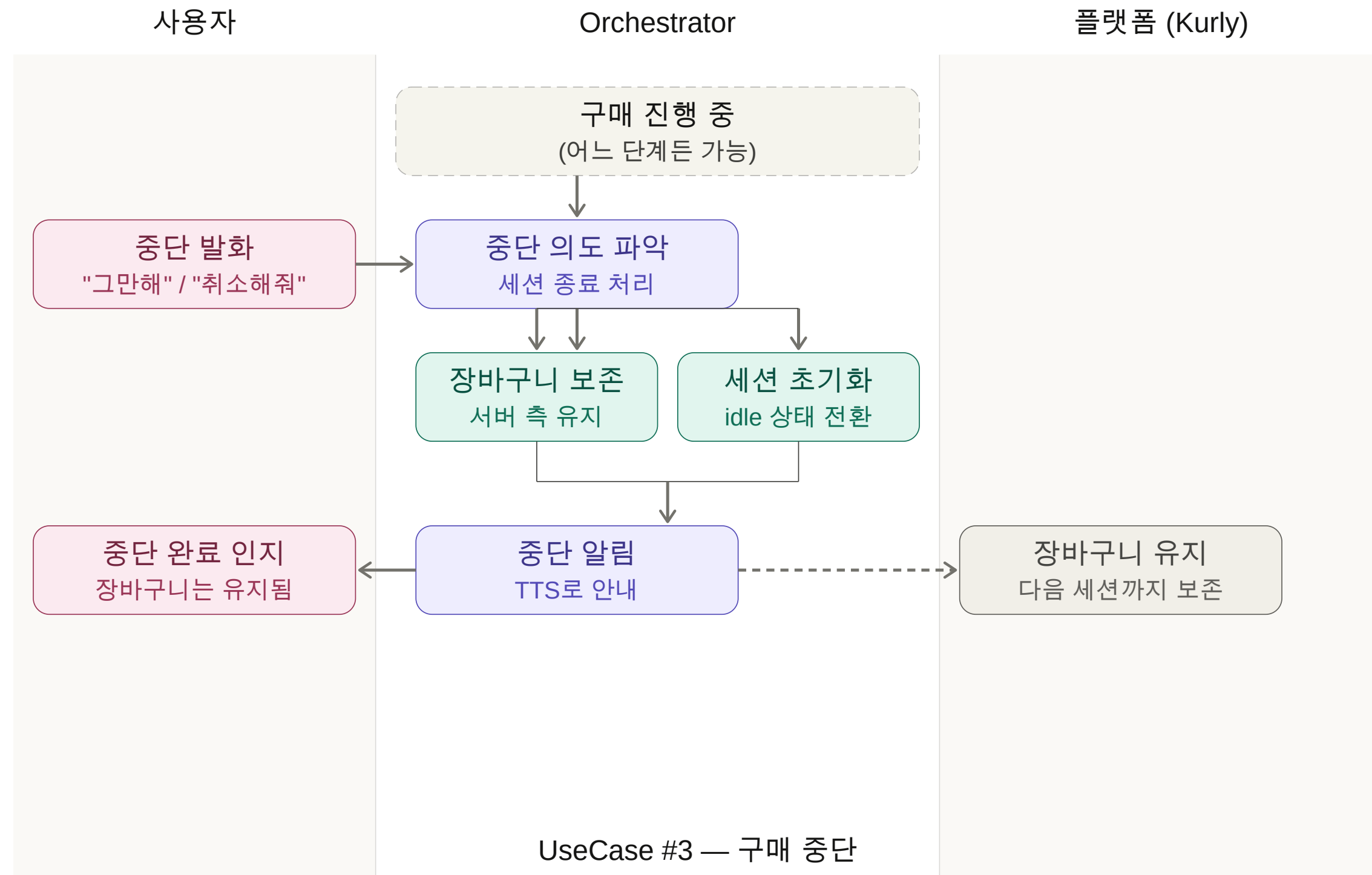


UseCase #2 — 재구매

Case #3 : 구매 중 중단

주문하다가 마음 바뀌면?
언제든지 말만하세요!

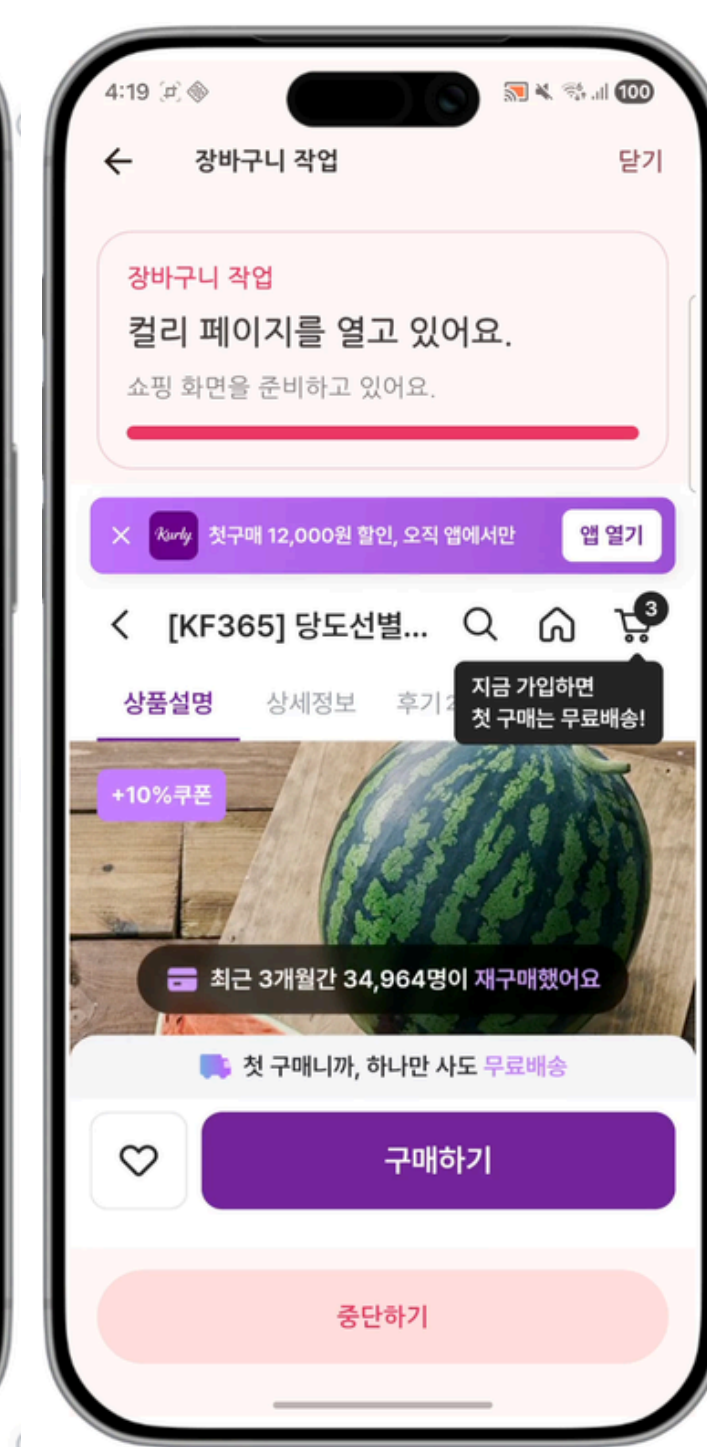
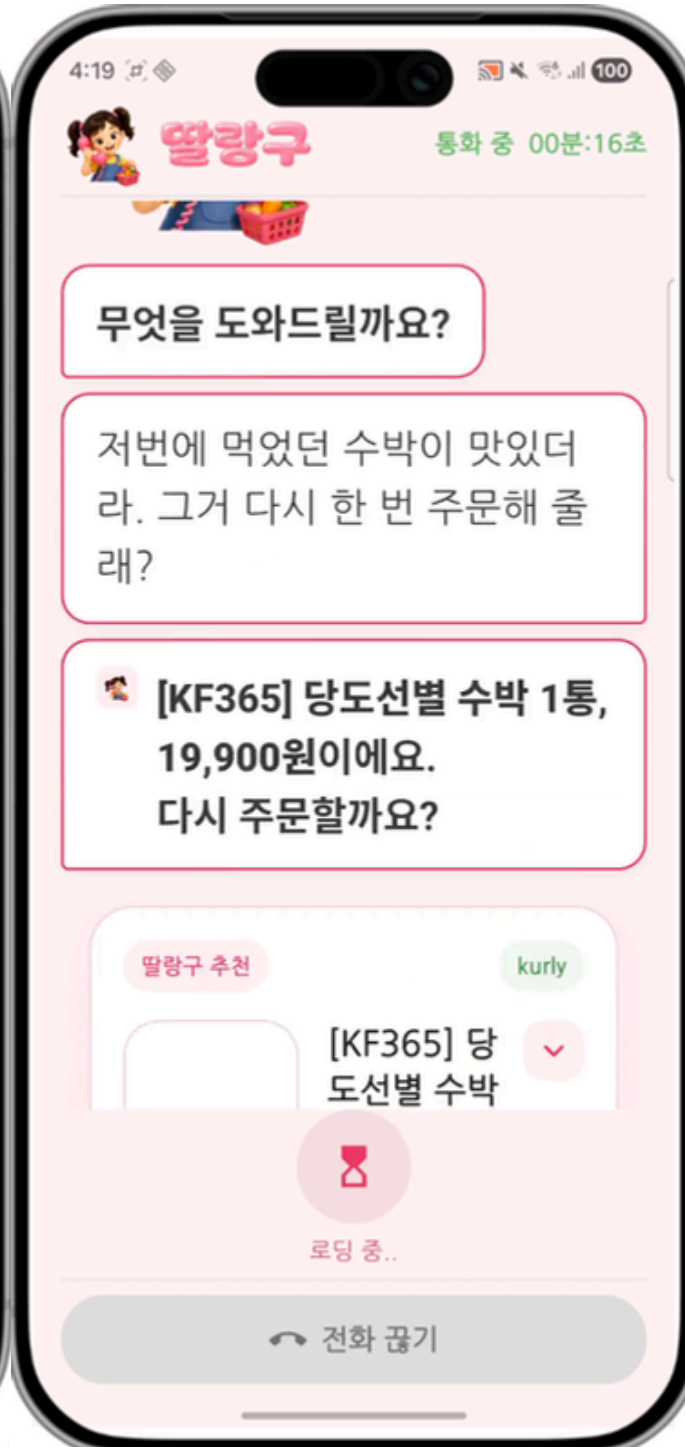
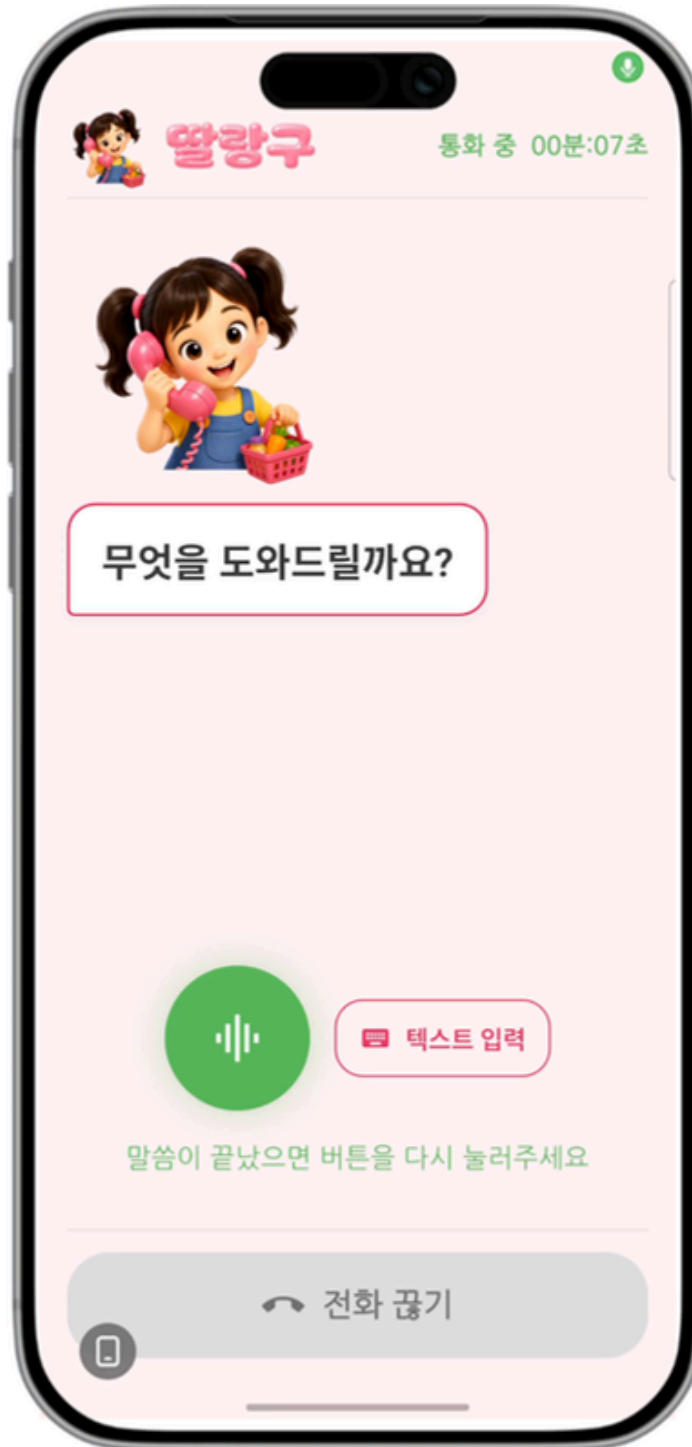
달랑구가 해결!



UseCase #3 — 구매 중단

03

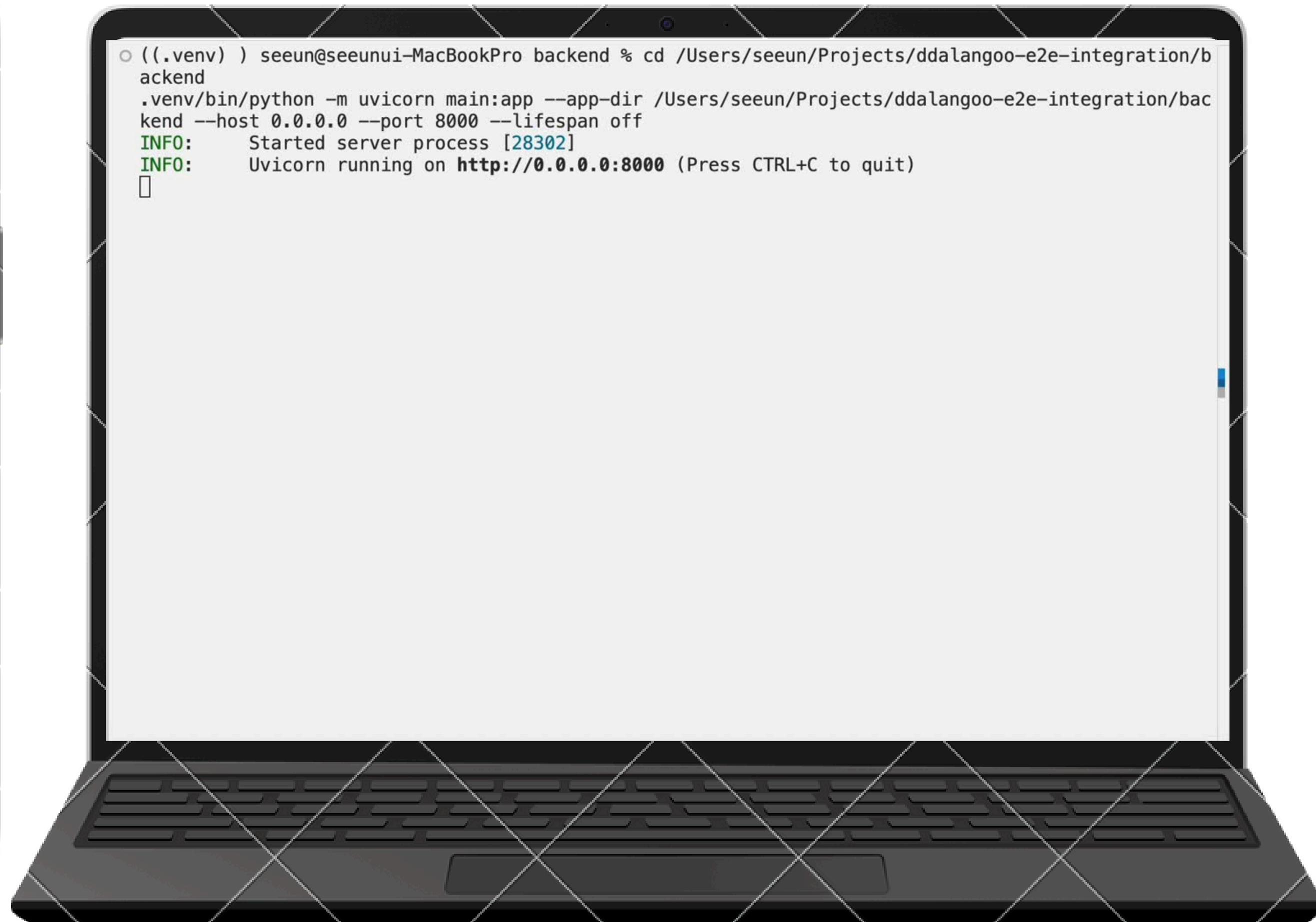
DEMO



Front

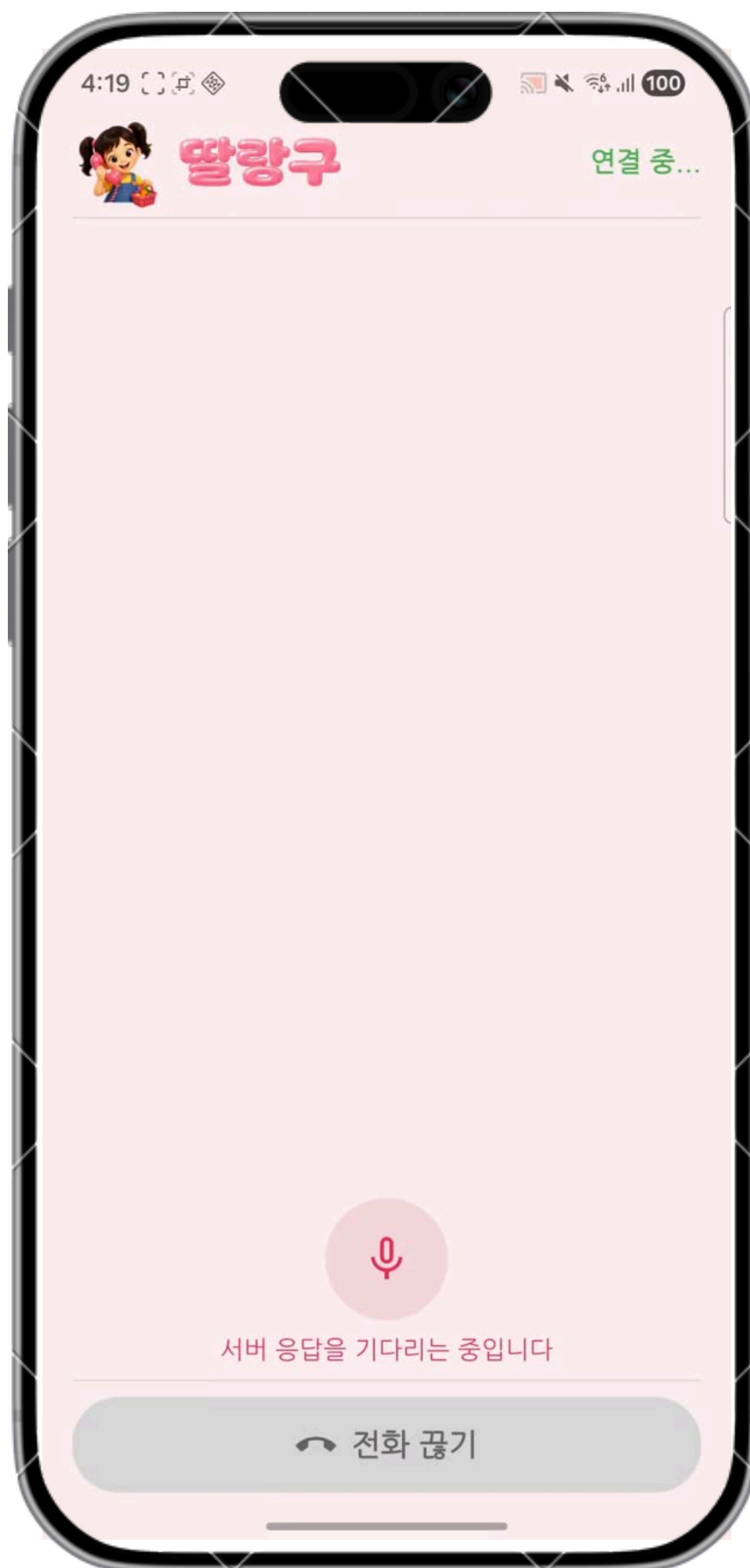


Back

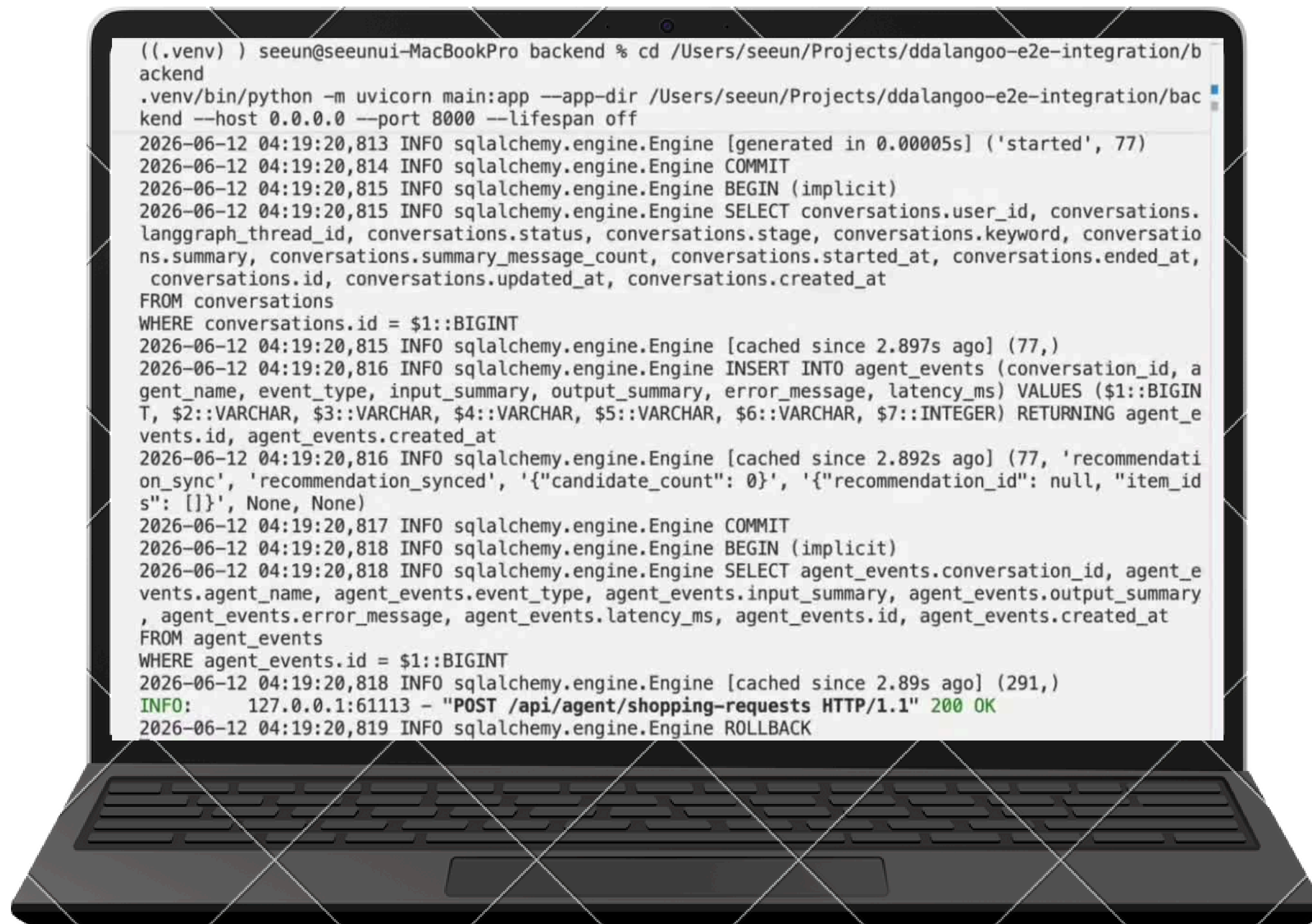


CASE #2

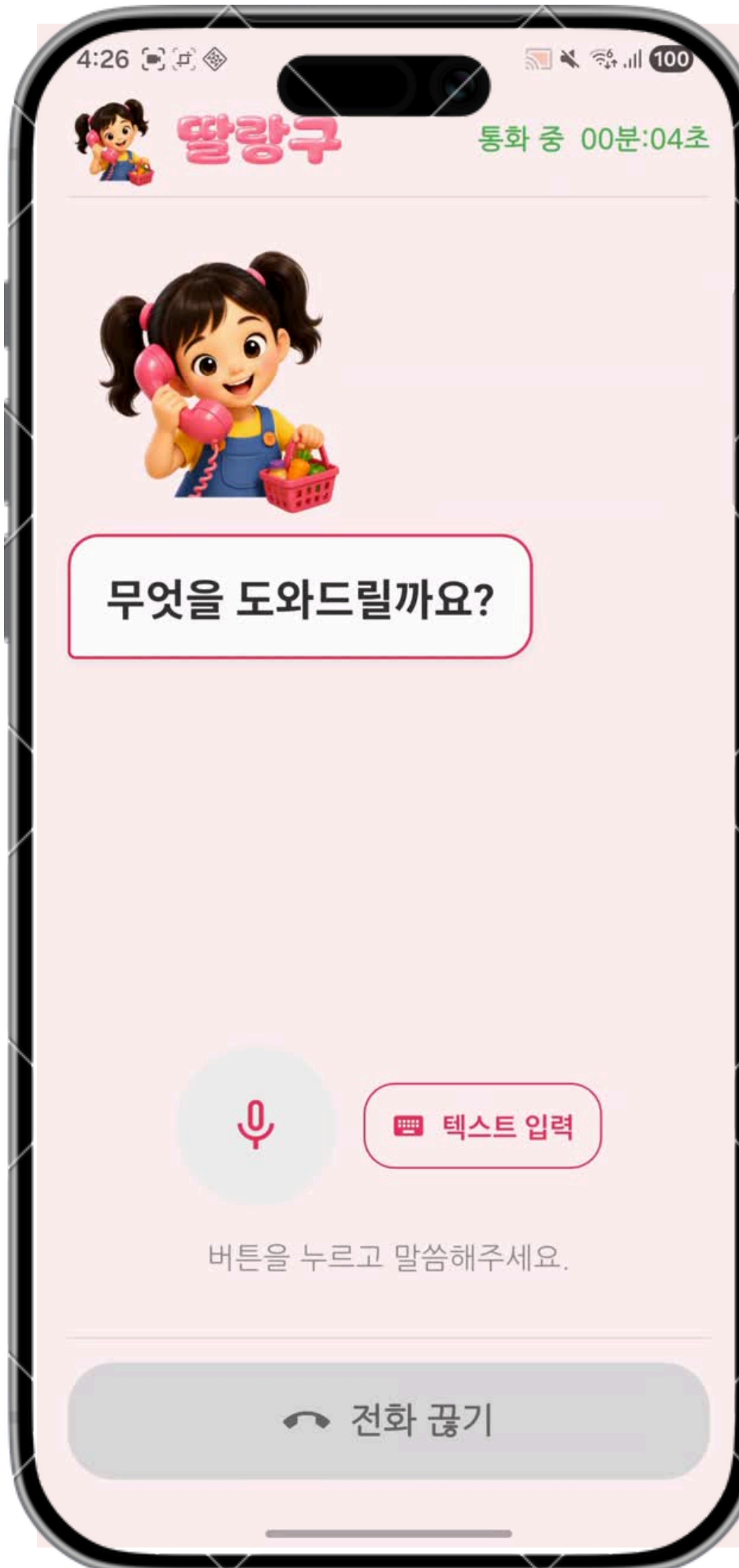
Front



Back



Front



CASE #3

Back

```

WHERE conversations.id = $1::BIGINT
2026-06-12 04:26:11,589 INFO sqlalchemy.engine.Engine [cached since 409s ago] (84,)
2026-06-12 04:26:11,589 INFO sqlalchemy.engine.Engine UPDATE conversations SET status=$1::VARCHAR,
updated_at=now() WHERE conversations.id = $2::BIGINT
2026-06-12 04:26:11,589 INFO sqlalchemy.engine.Engine [cached since 409s ago] ('started', 84)
2026-06-12 04:26:11,589 INFO sqlalchemy.engine.Engine COMMIT
2026-06-12 04:26:11,590 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-06-12 04:26:11,590 INFO sqlalchemy.engine.Engine SELECT conversations.user_id, conversations.
langgraph_thread_id, conversations.status, conversations.stage, conversations.keyword, conversatio
ns.summary, conversations.summary_message_count, conversations.started_at, conversations.ended_at,
conversations.id, conversations.updated_at, conversations.created_at
FROM conversations
WHERE conversations.id = $1::BIGINT
2026-06-12 04:26:11,590 INFO sqlalchemy.engine.Engine [cached since 411.9s ago] (84,)
2026-06-12 04:26:11,590 INFO sqlalchemy.engine.Engine INSERT INTO agent_events (conversation_id, a
gent_name, event_type, input_summary, output_summary, error_message, latency_ms) VALUES ($1::BIGIN
T, $2::VARCHAR, $3::VARCHAR, $4::VARCHAR, $5::VARCHAR, $6::VARCHAR, $7::INTEGER) RETURNING agent_e
vents.id, agent_events.created_at
2026-06-12 04:26:11,591 INFO sqlalchemy.engine.Engine [cached since 411.9s ago] (84, 'recommendati
on_sync', 'recommendation_synced', '{"candidate_count": 0}', '{"recommendation_id": null, "item_id
s": []}', None, None)
2026-06-12 04:26:11,591 INFO sqlalchemy.engine.Engine COMMIT
2026-06-12 04:26:11,591 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-06-12 04:26:11,592 INFO sqlalchemy.engine.Engine SELECT agent_events.conversation_id, agent_e
vents.agent_name, agent_events.event_type, agent_events.input_summary, agent_events.output_summary
, agent_events.error_message, agent_events.latency_ms, agent_events.id, agent_events.created_at
FROM agent_events
WHERE agent_events.id = $1::BIGINT
2026-06-12 04:26:11,592 INFO sqlalchemy.engine.Engine [cached since 411.9s ago] (343,)
INFO:      127.0.0.1:61233 - "POST /api/agent/shopping-requests HTTP/1.1" 200 OK
2026-06-12 04:26:11,592 INFO sqlalchemy.engine.Engine ROLLBACK

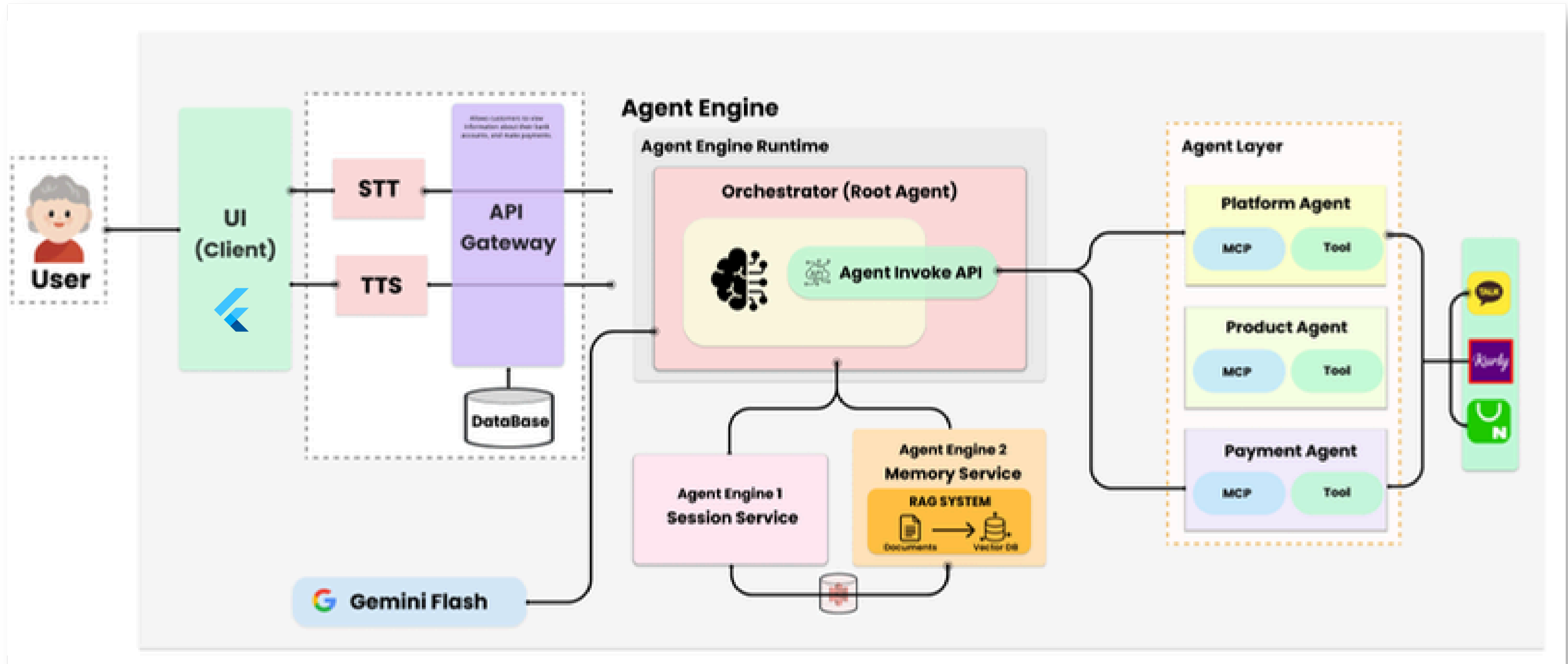
```


04

Technical Details

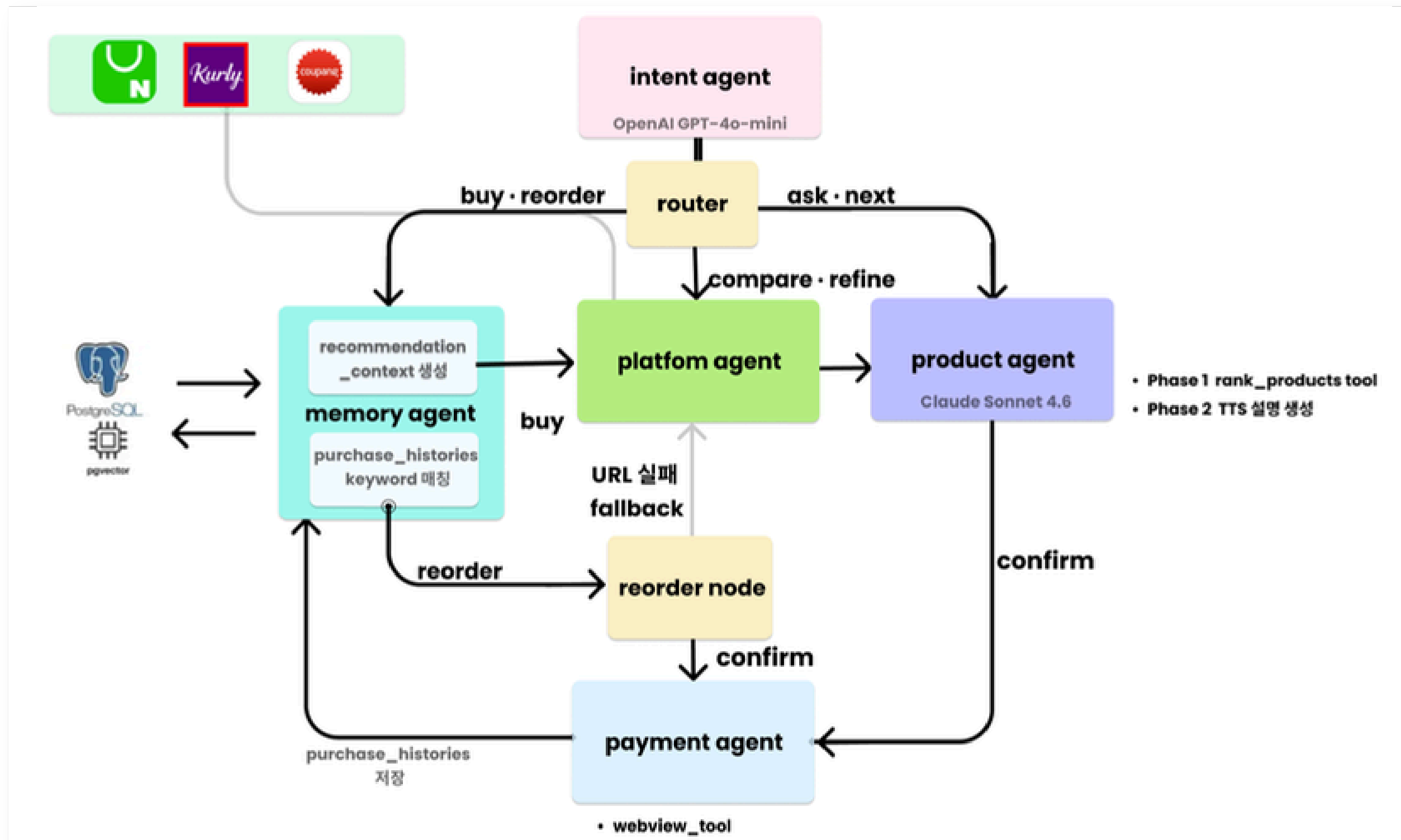
System Architecture

- 전체 시스템 아키텍처는 중간발표 때와 동일함.



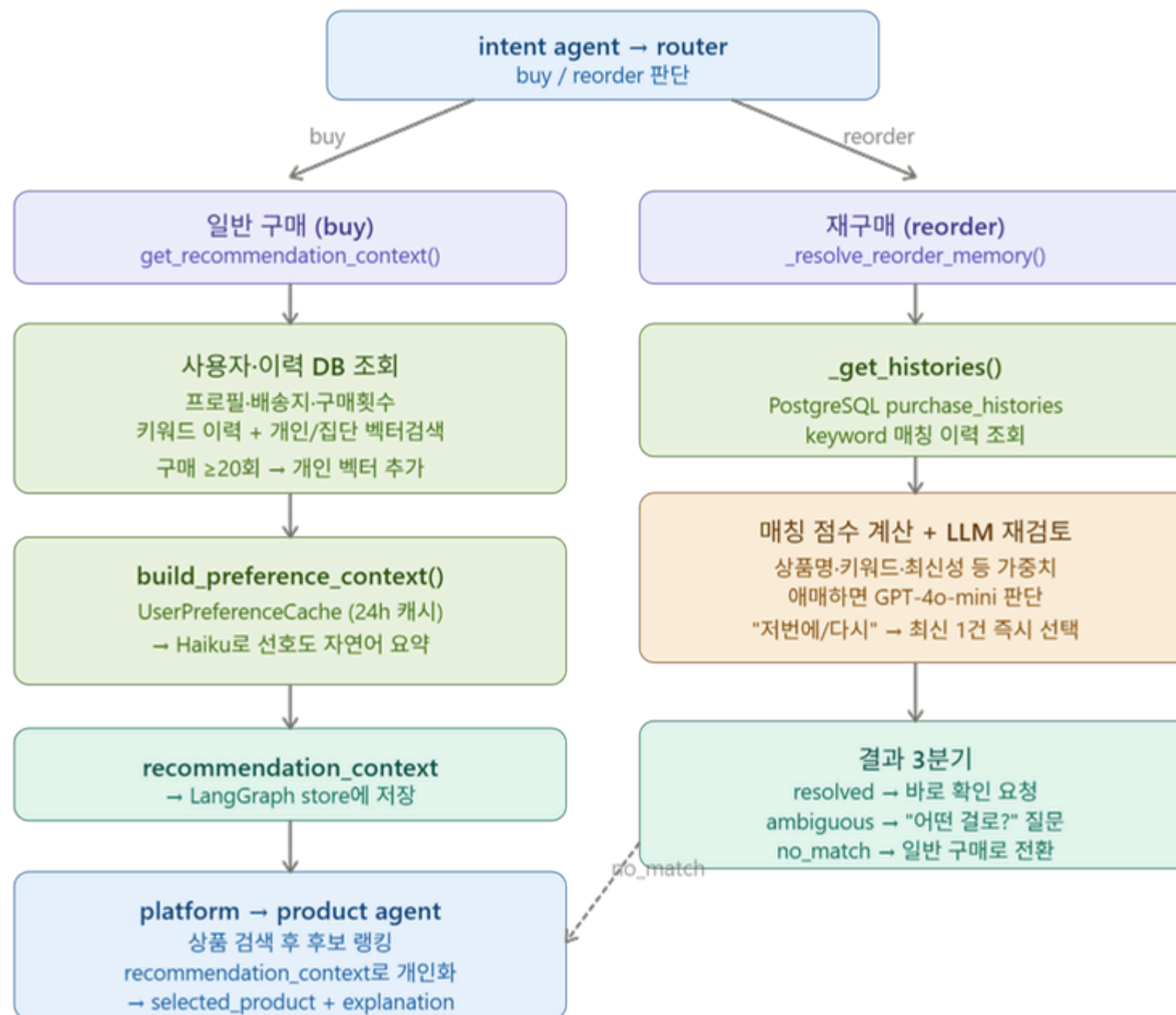
Multi-Agent 구조

- Memory Agent가 제공하는 과거 이력을 기반으로 각 Agent가 유기적으로 연결되어 검색부터 결제까지 자동으로 이어짐



추천 로직 설계

- 구매이력 기반 선호도를 JSONB 캐시로 저장해서 추천 시 재사용
 - memory agent가 계산 → product/platform agent에 전달



1. general 캐시

- 선호 브랜드, 가격대, 재구매 패턴, 선호 플랫폼
- TTL 24시간, 구매 완료시 무효화

2. keyword 캐시

- "딸기" 검색 시 과거 구매 이력만 모아서 저장



User Preference Cache (예시)

추천 로직 설계

- product agent 내 추천 알고리즘

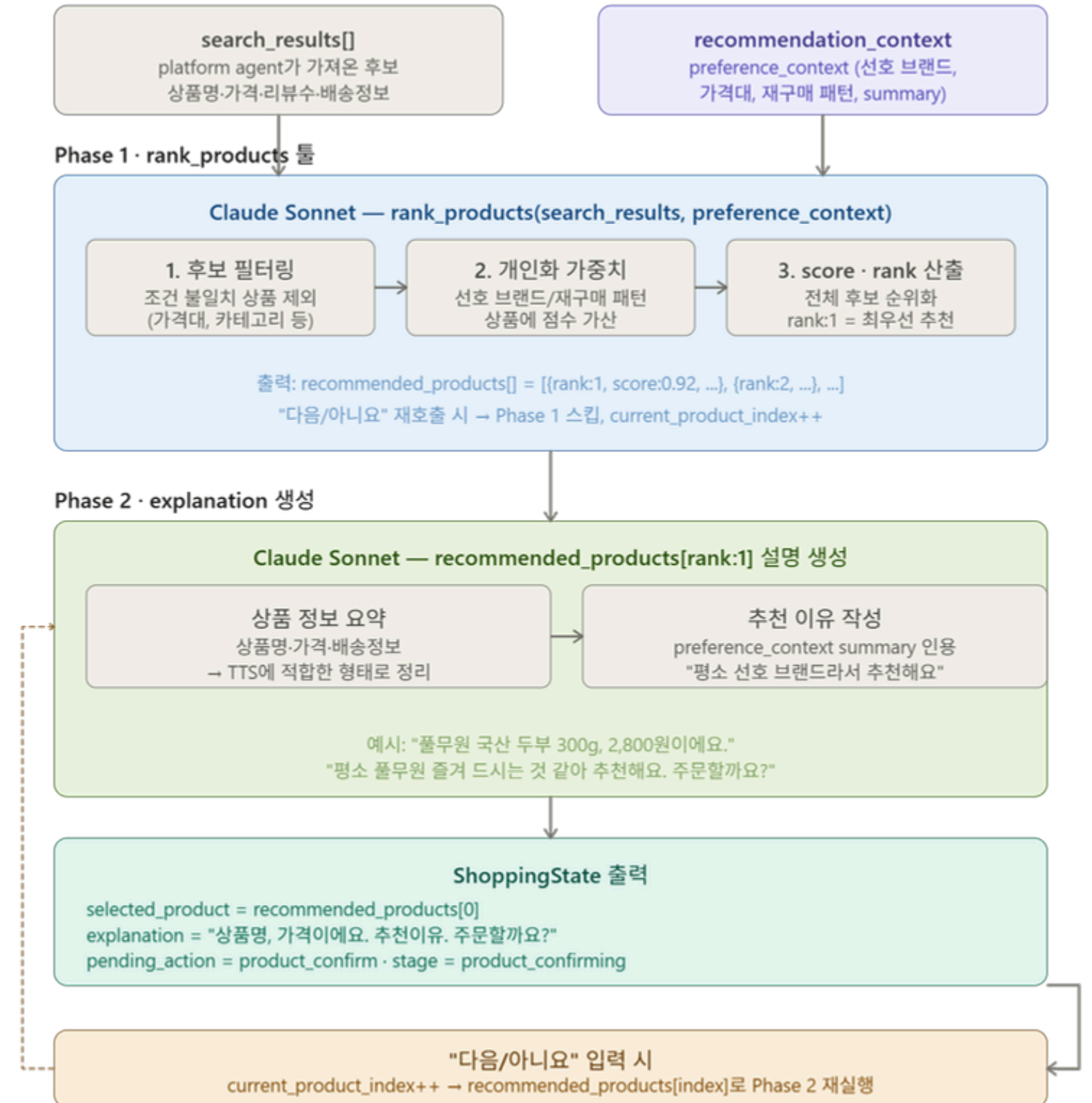
Phase1. Product Ranking

- search_results + preference_context를 입력으로 Claude Sonnet이 후보 필터링
- 개인화 가중치 → score 산출 후 전체 순위화
- rank:1을 최우선 추천으로 선정

Phase2. Explanation

- rank:1 상품에 대해 상품 정보 요약 + 추천 이유 생성
- "다음" 요청 시 재랭킹 없이, current_product_index만 증가시켜 Phase2 재실행

- 결과는 ShoppingState로 기록되어 router의 다음 분기 판단에 사용됨



LangGraph 설계

- State 중심 설계

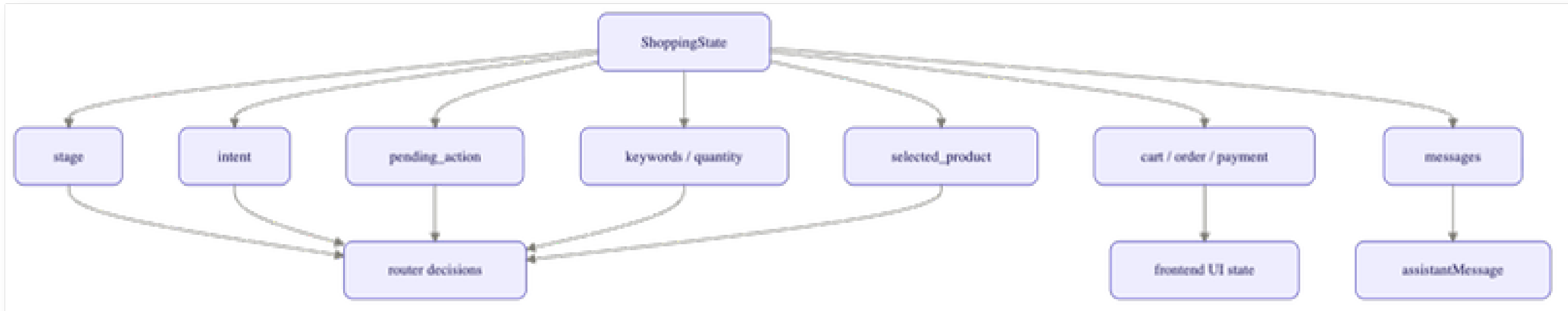
- 단일 LLM 호출이 아닌, ShoppingState를 중심으로 Agent들이 협업

1. ShoppingState (Main Graph) — 대화 전체의 단일 작업 메모리

2. PaymentState — 결제 단계만 분리해 안전하게 관리

3. MemoryState — 구매 이력/선호도 등 장기 문맥

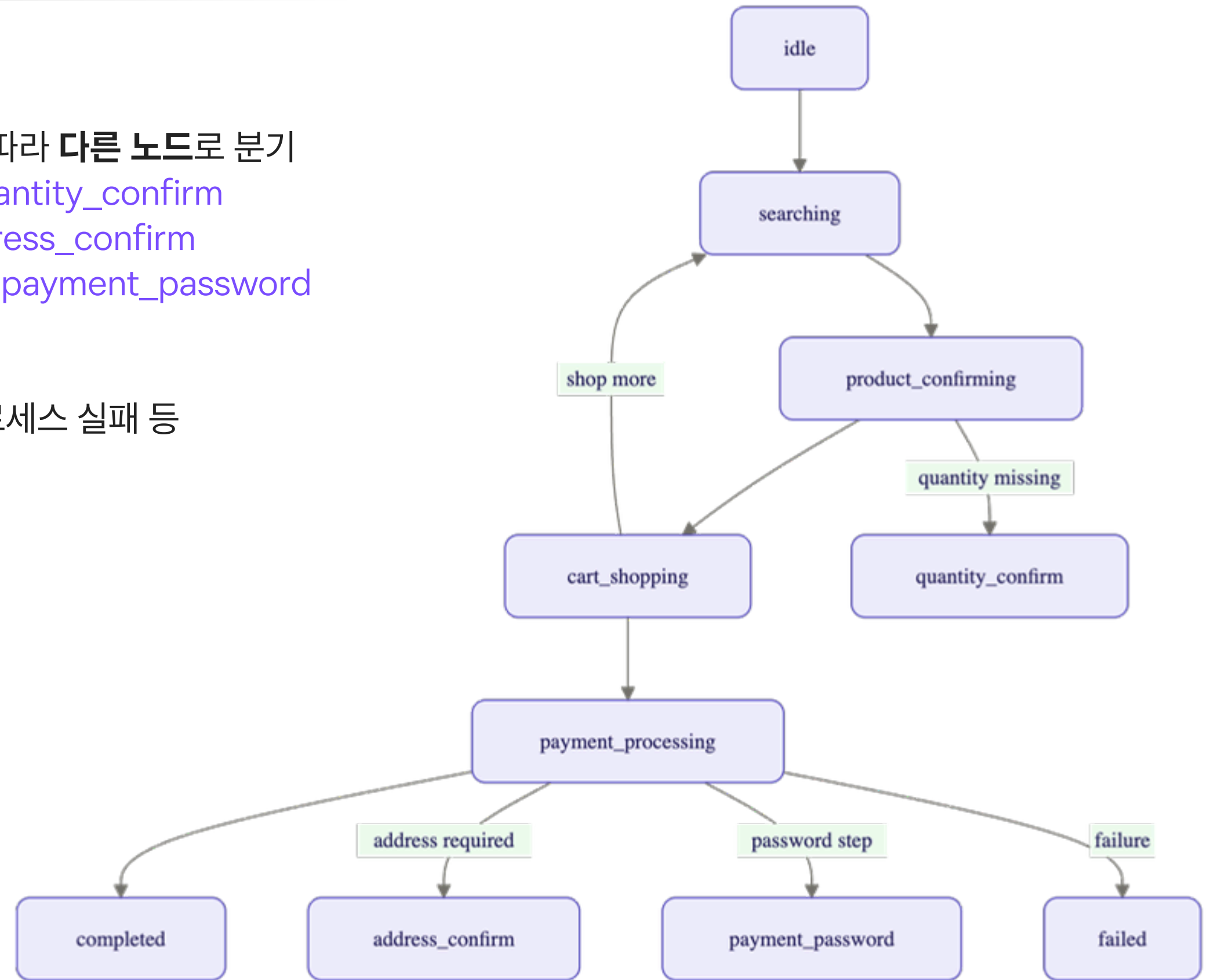
- ShoppingState



LangGraph 설계

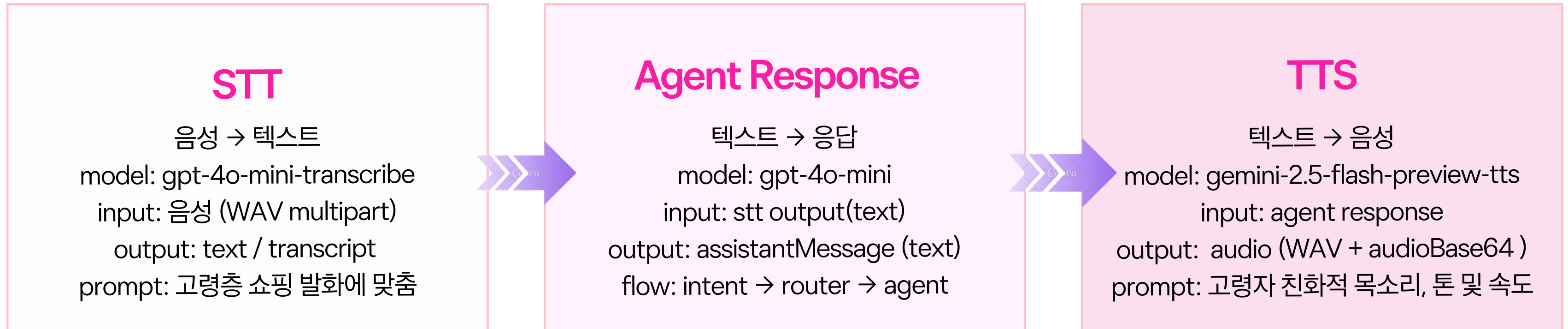
● State Flow

- 같은 'idle → completed' 경로도 중간 state 값에 따라 **다른 노드**로 분기
 - product_confirming + quantity 없음 → quantity_confirm
 - payment_processing + 배송지 없음 → address_confirm
 - payment_processing + 배송지 확인 완료 → payment_password
 - payment_processing + 결제 실패 → failed
- 재구매, 수량 체크, 배송지 확인, 비밀번호 확인, 프로세스 실패 등
- 다양한 state에서의 안정적인 처리 설계



음성 처리 파이프라인

- Agent architecture + voice input/output → end-to-end voice shopping assistant



```
# 쇼핑 도메인 표현 정확도를 높이기 위한 STT 프롬프트.
_STT_PROMPT = (
    "너는 한국어 음성 인식 엔진이다. "
    "오디오에서 실제로 들리는 사용자의 말만 한 줄 한국어 텍스트로 전사해라. "
    "오디오에 없는 문장, 대화 예시, 답변, 설명은 절대 만들지 마라. "
    "상품명, 수량, 가격, 배송지, 장바구니, 결제, 재주문과 관련된 표현은 특히 정확히 전사해라. "
    "말이 불명확하거나 배경음/무음이면 빈 문자열만 출력해라. "
    "줄바꿈 없이 텍스트만 출력해라."
)

_MAX_TRANSCRIPT_LENGTH = 200
_MIN_AUDIO_BYTES = 1000 # 너무 작은 파일은 무음으로 간주
_MAX_AUDIO_BYTES = 20 * 1024 * 1024 # 20 MB
```

```
_TTS_VOICE = os.getenv("GEMINI_TTS_VOICE", "Zephyr")
_TTS_SAMPLE_RATE = 24000

def _tts_prompt(text: str) -> str:
    return (
        "Read the exact following Korean text in Korean. "
        "Speak like a warm, affectionate daughter helping an older parent shop. "
        "Use a bright, reassuring, and very kind tone. "
        "Keep the voice gentle, patient, and easy for older adults to understand. "
        "Speak about 1.2x faster than a neutral default pace without sounding rushed. "
        "Pause naturally between sentences. "
        "Pronounce prices, quantities, dates, addresses, and payment-related words very clearly. "
        "Sound friendly and comforting, never cold or robotic. "
        "Do not add, remove, or change any words.\n"
        f"{text}"
    )
```

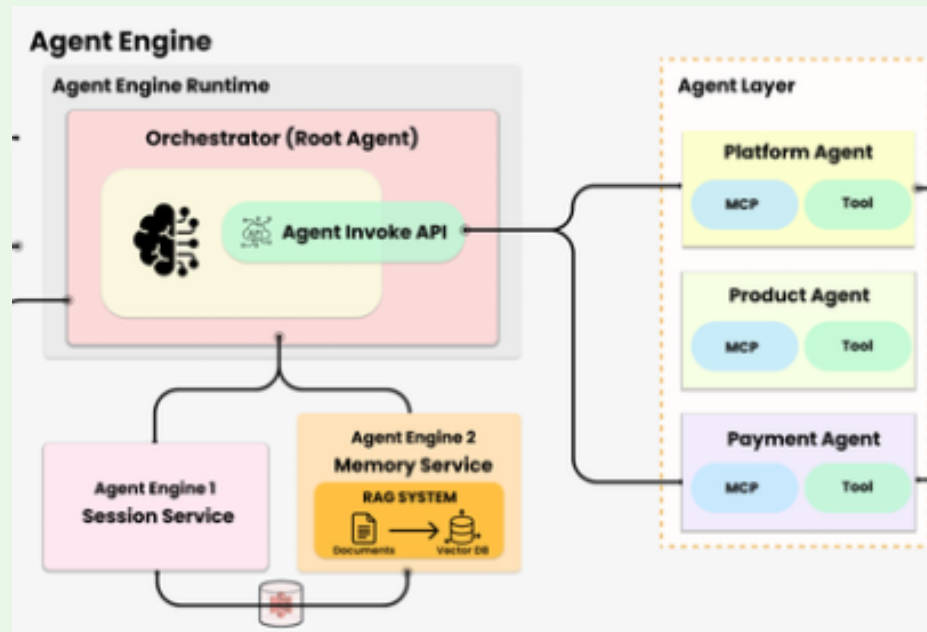
06

Challenges & Solutions

크게 3개 파트에서

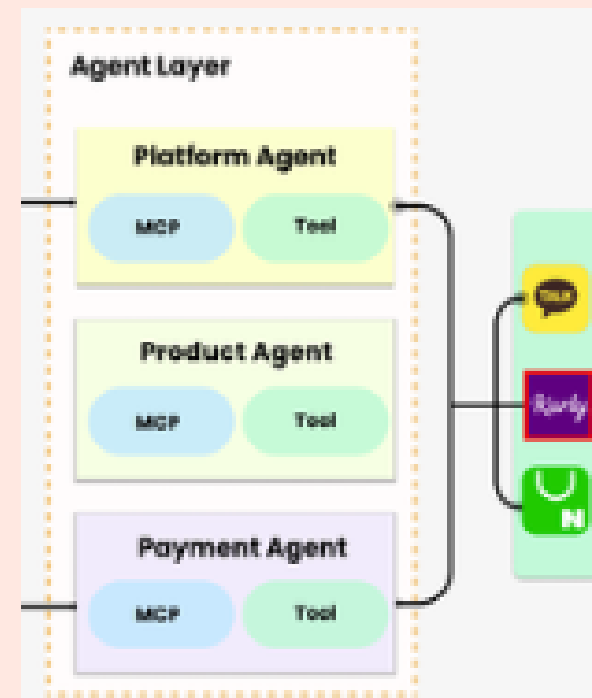
- 본 시스템의 성능을 좌우하는 파트는 크게 3가지로, Agent System / Webview (Kernel Level) / STT-TTS 이다.

Agent System



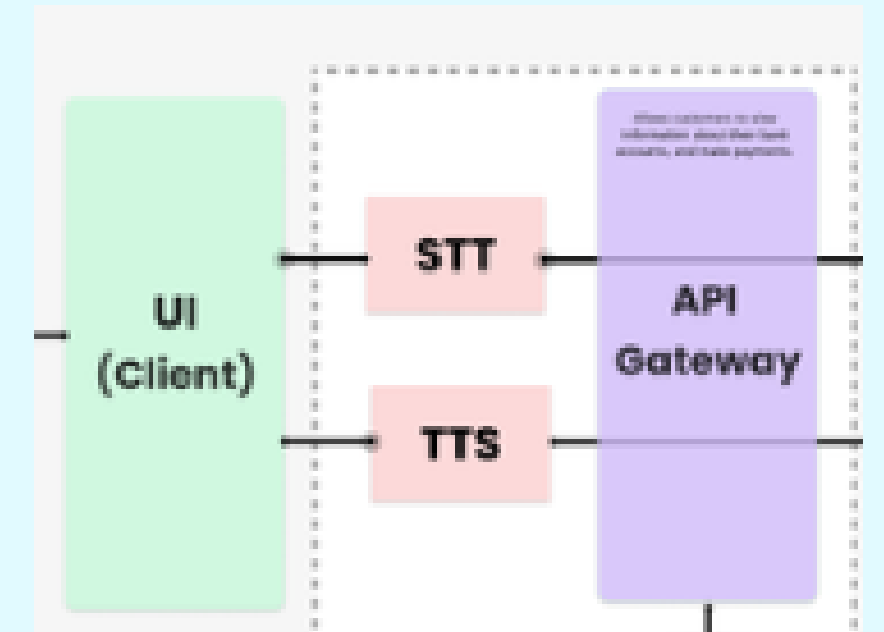
Agent가 우리의 의도대로
추론하는가?

Webview (Kernel Level)



Agent의 응답을 실시간으로
정확히 렌더링하는가?

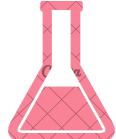
STT/TTS

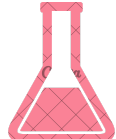


정확도와 속도가
상용 가능화한 수준인가?

Part 1. Agent System

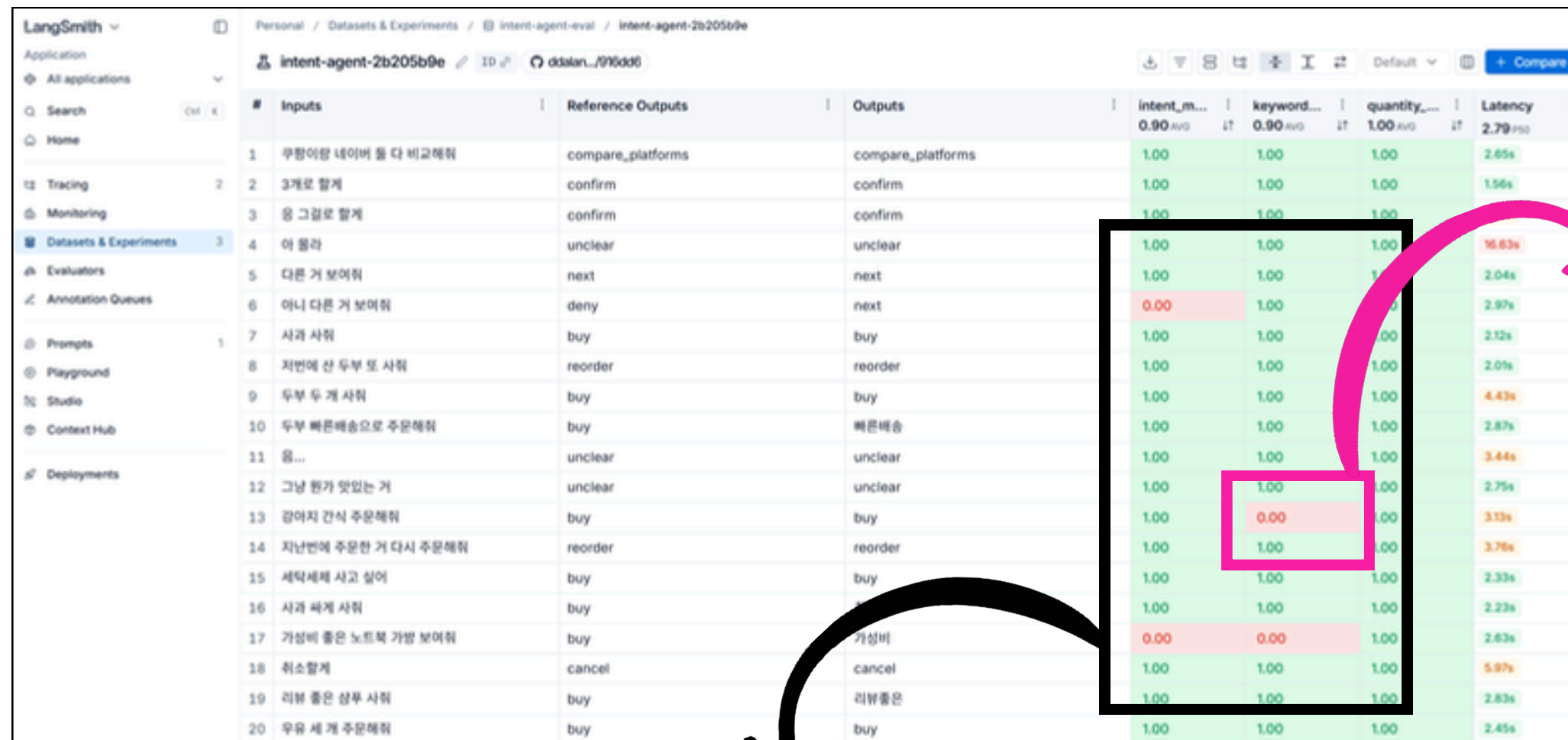
- Agent 구현 후, Langsmith를 활용해 실험을 진행함

 실험 1. Intent agent가 **의도, 키워드 추출**을 잘 해내는가?

 실험 2. Product Agent가 적절하게 **추천**을 잘 하는가?

LangSmith 기반 에이전트 성능 분석 (1)

- 실험 1. Intent Agent가 intent 판단을 잘 해내는가? (문제 파악)



#	Inputs	Reference Outputs	Outputs	intent_m...	keyword...	quantity...	Latency
				0.90 avg	0.90 avg	1.00 avg	2.79 ms
1	쿠팡이랑 네이버 둘 다 비교해줘	compare_platforms	compare_platforms	1.00	1.00	1.00	2.65s
2	3개로 할게	confirm	confirm	1.00	1.00	1.00	1.56s
3	응 그걸로 할게	confirm	confirm	1.00	1.00	1.00	
4	어 몰라	unclear	unclear	1.00	1.00	1.00	16.63s
5	다른 거 보여줘	next	next	1.00	1.00	1.00	2.04s
6	아니 다른 거 보여줘	deny	next	0.00	1.00	1.00	2.97s
7	사과 사줘	buy	buy	1.00	1.00	1.00	2.12s
8	저번에 산 두부 또 사줘	reorder	reorder	1.00	1.00	1.00	2.01s
9	두부 두 개 사줘	buy	buy	1.00	1.00	1.00	4.43s
10	두부 빠른배송으로 주문해줘	buy	빠른배송	1.00	1.00	1.00	2.87s
11	음...	unclear	unclear	1.00	1.00	1.00	3.44s
12	그냥 뭔가 맛있는 거	unclear	unclear	1.00	1.00	1.00	2.75s
13	강아지 간식 주문해줘	buy	buy	1.00	0.00	1.00	3.13s
14	지난번에 주문한 거 다시 주문해줘	reorder	reorder	1.00	1.00	1.00	3.76s
15	세탁제 사고 싶어	buy	buy	1.00	1.00	1.00	2.33s
16	사과 세 개 사줘	buy	사과	1.00	1.00	1.00	2.23s
17	가성비 좋은 노트북 가방 보여줘	buy	가성비	0.00	0.00	1.00	2.63s
18	취소할게	cancel	cancel	1.00	1.00	1.00	5.97s
19	리뷰 좋은 삼투 사줘	buy	리뷰좋은	1.00	1.00	1.00	2.83s
20	우유 세 개 주문해줘	buy	buy	1.00	1.00	1.00	2.45s

[EXAMPLE CASE]

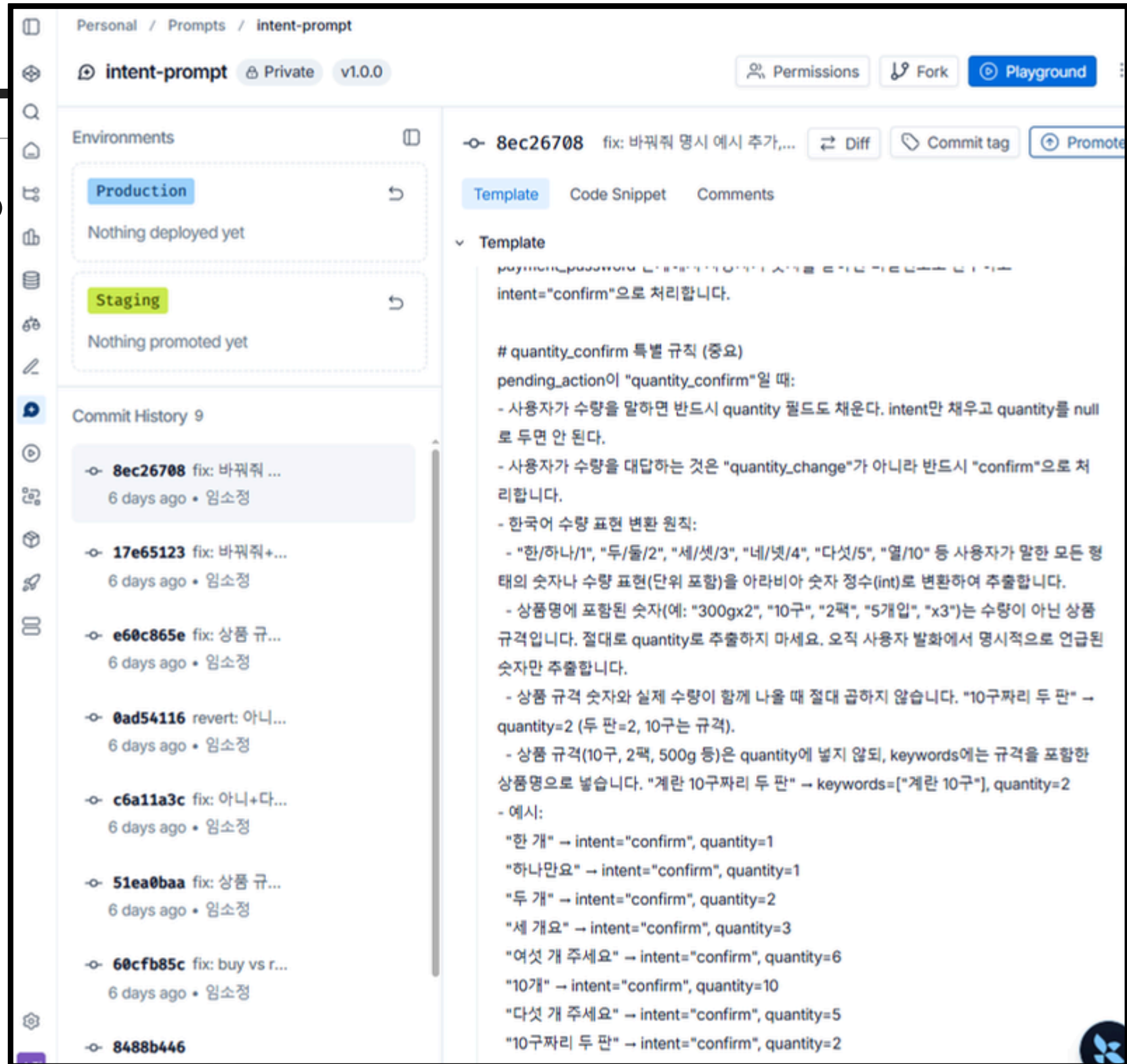
Query : "강아지 간식 사줘"

GT : "강아지 간식"

AGENT: "강아지", "간식" 으로 추출

- 실험용 데이터셋을 구축하고,
- Intent Agent가 각 여러 종류의 쿼리가 들어왔을 때 응답을 잘 반환하는 지를 테스트
- 대부분의 쿼리에 대해 응답을 잘 반환하였으나 일부 오류를 발견. (정답률 83.3%)

석 (1)



위와 같은 문제들을 모두 파악하고, 프롬프트를 강화함

모델 출력 (오류)

알아줘"

의도로 refine을 출력해야 하는데 buy를 출력함

quantity = 2 (10구는 규격이므로) 로 예측해야 하는데
20으로 예측함

데이터 셋 정답을 next가 아닌 deny로 설정해둠

'계란 10구'로 키워드 추출해야 하는데 '계란'만 추출함

quantity_change 의도의 발화를 confirm으로 출력

초기 AGENT 정확도

83.3 %

▲ +15.6%

개선 후 정확도

99.9 %

LangSmith 기반 에이전트 성능 분석 (2)

● 실험 2. Product Agent가 적절하게 추천을 잘 하는가?



case A) **브랜드 선호도** 기반으로
쿼리가 들어왔을 때,

이를 잘 추천하는지 확인

```
Output
Fields
  explanation
    풀무원 국산콩 두부 400g, 2,800원이에요. 평소 즐겨 사시는 브랜드예요. 주문할까요?
  preference_keywords
    두부: 풀무원 선호, 가격대 2000~4000원
  preference_summary
    풀무원 제품을 반복 구매한 이력이 있음
  rank1_product
    풀무원 국산콩 두부 400g
  all_ranked_products
    0 풀무원 국산콩 두부 400g
    1 CJ 행복한콩 두부 300g
  candidates_count
    3
```

```
search_results
  0
    delivery 일반배송
    platform naver
    product_name CJ 행복한콩 두부 300g
    product_url https://c.com/1
    price 1500
    rating 4.2
    review_count 210
  1
    delivery 로켓배송
    platform coupang
    product_name 풀무원 국산콩 두부 400g
    product_url https://c.com/2
    price 2800
    rating 4.7
    review_count 3200
  2
    delivery 일반배송
    platform naver
    product_name 자연두부 순두부 350g
    product_url https://c.com/3
```

case B) **가격 선호도** 기반으로
쿼리가 들어왔을 때,

이를 잘 추천하는지 확인

```
Output
Fields
  explanation
    국산 달걀 30구, 3,900원이에요. 후보 중 가장 저렴해요. 주문할까요?
  preference_keywords
  preference_summary
    선호 정보 없음 (구매이력 부족)
  rank1_product
    국산 달걀 30구
  all_ranked_products
    0 국산 달걀 30구
    1 풀무원 목초란 10구
    2 동물복지 유정란 15구
  candidates_count
    3
  Reference Outputs
    Fields
      expected_rank1_product
        국산 달걀 30구
```

```
search_results
  0
    delivery 로켓배송
    platform coupang
    product_name 풀무원 목초란 10구
    product_url https://a.com/1
    price 4200
    rating 4.6
    review_count 1800
  1
    delivery 일반배송
    platform naver
    product_name 동물복지 유정란 15구
    product_url https://a.com/2
    price 6800
    rating 4.8
    review_count 920
  2
    delivery 일반배송
    platform naver
    product_name 국산 달걀 30구
    product_url https://a.com/3
    price 3900
```

Part2. Webview 동작 정확도 개선

- Webview 처리를 프론트엔드로 가져오면서 운영 안정성을 높임

BEFORE

백엔드 Playwright WebView

- 제어력은 높지만, 배포 환경에서 브라우저 런타임 의존성 ↑
- 세션을 복원 시 배포 환경(서버)과 디바이스/IP가 사용자와 달라 봇 탐지에 취약

배포 환경에서 브라우저 실행 안정성, 접근성,
로그인/세션 처리 등의 운영 문제



AFTER

프론트엔드 앱 내 WebView

- 대화 의사결정 계층은 백에 유지하고, 브라우저 실행 계층을 프론트로 이동
- 앱 내 WebView 제어 방식 : DOM을 읽고 클릭
- 앱 내 secure storage에 자격 정보를 저장 → 정보 접근 용이

실제 사용자 세션과 디바이스 환경을
그대로 활용 → 운영 안정성↑

Part3. STT/TTS

- STT 인식률, 속도를 위해 여러 STT 모델과 프롬프팅 시도
- 응답 속도 개선 + 할루시네이션(없는 문장 생성) 감소를 위해 모델 전환

BEFORE

gemini-2.5-flash-preview

- 상대적으로 느린 응답 속도
- 느린 응답 속도 대비 좋지 않은 인식률
- 무음/짧은 발화에서도 임의의 텍스트 생성되는 경우



AFTER

gpt-4o-mini-transcribe

- 응답 속도 개선 → 음성 인터랙션 체감 지연 감소
- 무음 시 빈 문자열 반환
- 도메인 특화 프롬프트로 쇼핑 용어 인식률 확보

속도와 정확도,
그리고 안정성 개선!